

Capstone Project, Part 1: Design Report

Leslie Montgomery

McMaster University

BDV 102: Interactivity and Databases

Per Pettersson

November 2025

Introduction

SwiftCart is a lightweight e-commerce project designed, with a backend focused on modularity, maintainability, and RESTful design. This report outlines the Entity Relationship Diagram (ERD), OpenAPI documentation, and database design rationale behind the SwiftCart system. The backend architecture is built using *Node.js* (Express), *PostgreSQL* (hosted via *Neon.tech*), and the API-first approach was enabled through *APIDog*. The ERD was modeled using *LucidChart*, and development was completed using *VS Code*.

ERD Design

The Entity Relationship Diagram (ERD) provides a high-level view of the core business entities and the relationships between them. Some key entities include:

- **User**
- **Product**
- **Category**
- **CartItem**
- **Order**
- **OrderItem**
- **Address**
- **PaymentMethod**
- **WishlistItem**

Each table is linked with foreign keys so the data stays consistent. For example, a Product must belong to a valid Category, and an Order must belong to a specific User. I designed the structure to be in **Third Normal Form (3NF)** to avoid unnecessary duplication and allow the system to scale as more features are added.

API Design

The API was designed using the *OpenAPI 3.0.3* standard in *APIDog*, inspired by a design-first approach. RESTful principles guided endpoint naming and structure, following conventions like

/products, /cart, /checkout, and /orders. This helps standardize communication between the backend and frontend and makes the API easy to test.

The API follows typical RESTful design choices, with routes structured like:

- **GET /products:** Lists available products
- **GET /cart:** Retrieves the current user's cart
- **POST /cart:** Adds a product item to the cart
- **POST /checkout:** Converts the user's cart into an order
- **GET /orders:** Retrieves user's order history

Each endpoint returns JSON responses with appropriate status codes.

Design Decisions

The database schema was designed according to **3NF (Third Normal Form)** standards to eliminate redundancy and achieve consistency for possible future scaling. The use of foreign keys between tables enforces referential integrity (e.g., each product must belong to a valid category).

Key Decisions:

- Instead of creating a separate Cart table, I used a **CartItem** table to represent the user's current cart. Each row links a product to a user, and I ensure the same product isn't added twice; instead, the quantity is updated, to avoid duplication.
- I created an **Address** table so that users can save multiple addresses (for billing and shipping). This table includes a type field so the system knows which address to use during checkout.
- For wishlists, I created a **WishlistItem** table. These entries don't need a quantity because wishlists usually just track interest in products, not how many the user plans to buy.
- I separated **OrderItem** from Order so each order can contain multiple products. This is important for online stores because someone might buy several things in one order.

Schema Iteration Note

When I first designed the database, I included a Cart table and stored a single address directly on the User table. But after some further research and testing in LucidChart, I simplified the approach by:

- Removing the Cart table and using **CartItem** to represent the active cart per user.
- I separated shipping and billing addresses into a standalone **Address** table with a type attribute, enabling users to manage both independently.
- Removing a quantity field from the wishlist, because it didn't make sense for this use case.

These changes made the ERD more consistent and easier to understand.

Challenges & Trade-offs

One challenge was finding the balance between simplicity and flexibility. At first, I wanted to avoid too many tables, but I realized that separating things like addresses and order items would make it easier to scale later.

Another trade-off is that calculating things like order totals or cart previews will require queries at runtime rather than storing the values in the database. This makes the system more flexible but could slow it down in the future if it grows very large.

Summary

Overall, Part 1 of the SwiftCart project helped me practice planning and documenting a backend system using real tools like LucidChart and APIDog. My ERD, API, and schema design focus on being clean, scalable, and ready for development in the next phase. By using PostgreSQL and following normalization rules, I've set up a strong foundation that can support future features like authentication, and product reviews.